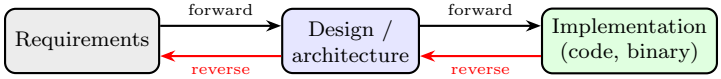
Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

Reverse Engineering, LLM Gateways and Proxies

1. Reverse Engineering (RE)

What it is: working **backwards from a finished system to understand how it works**, when its design, source code or documentation is not available. Forward engineering goes from idea → design → product; reverse engineering goes from product → design → idea.



Two main approaches:

- **Static analysis:** study the artifact without running it: read a binary's structure, disassemble it (machine code → assembly) or decompile it (assembly → approximate high-level code), inspect file formats and strings.
- **Dynamic analysis:** observe it while it runs: debuggers, logging system calls, watching memory, and capturing network traffic to infer an undocumented protocol or API.

Common tools: Ghidra (open source, by the NSA), IDA Pro, Binary Ninja, radare2 (disassembly / decompilation); gdb, x64dbg (debugging); Wireshark (network protocols); browser DevTools (web apps).

Legitimate uses: interoperability (making software talk to an undocumented format or protocol), maintaining legacy systems whose source code was lost, security research and malware analysis (defenders study how malicious code behaves to detect and block it), competitive analysis of hardware, and verifying what a product really does.

Reverse engineering in ML:

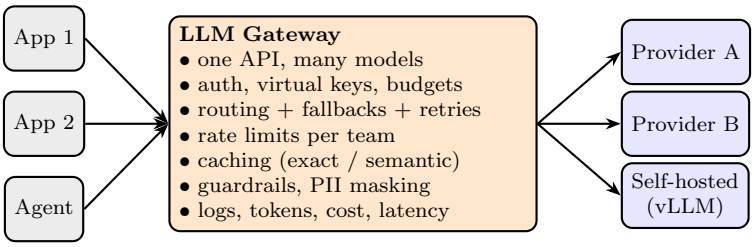
- **Mechanistic interpretability:** reverse engineering a trained neural network to discover which internal features and circuits produce a behavior, i.e. turning weights back into human-understandable algorithms.
- **Model extraction / distillation risk:** querying a model's API many times to train a copy that imitates it; providers limit this with terms of service, rate limits and monitoring.
- **Understanding a legacy pipeline:** reconstructing what an undocumented model or feature pipeline does from its inputs, outputs and code.

Legal and ethical note: legality depends on the country, the license (many EULAs forbid it) and the purpose. Interoperability and security research have some protections in many jurisdictions, while copying protected code, bypassing protections or cloning a product can be illegal. Always check the license and

get authorization.

2. LLM Gateway and other new terms

LLM Gateway: a **single entry point between your applications and all the LLM providers** you use (OpenAI, Anthropic, Google, Bedrock, self-hosted models). Applications call one API (usually OpenAI-compatible); the gateway handles everything around the call:



Why: avoid vendor lock-in (switch models by changing config, not code), keep running when a provider fails (automatic fallback), control and attribute spending per team, enforce security policies centrally, and get one place for observability. **Examples:** LiteLLM, Portkey, Kong AI Gateway, Cloudflare AI Gateway, OpenRouter, Envoy AI Gateway.

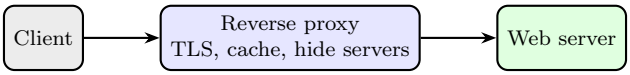
Other new terms you will hear:

Term	**Meaning**
Model routing	Choosing a model per request by difficulty, cost or latency: simple prompts to a small model, hard ones to a large one.
Semantic caching	Reusing a previous answer when a new prompt has a *similar meaning* (embedding similarity above a threshold), not just identical text.
Prompt caching	The provider reuses the computed KV cache of a repeated prompt prefix (system prompt, long document), lowering cost and latency.
Guardrails	Checks on inputs and outputs: prompt-injection detection, PII masking, toxicity and topic filters, output schema validation.
MCP (Model Context Protocol)	Open standard to connect LLM apps to tools and data sources through a common interface, like a "USB-C for tools".
A2A (Agent-to-Agent)	Protocol for agents from different systems to discover each other and delegate tasks.
Context engineering	Designing *everything* the model sees (instructions, retrieved data, tool results, memory, history), beyond writing a single prompt.
LLMOps / observability	Tracing every call (prompt, response, tokens, cost, latency) and evaluating quality over time; e.g. Langfuse, LangSmith, Arize Phoenix.
LLM-as-a-Judge	Using an LLM to grade outputs against a rubric, for evaluation at scale.
Structured outputs	Forcing the model to answer in a JSON schema, so responses are machine-parseable.
Agentic workflow	The LLM plans, calls tools, observes results and iterates in a loop until the task is done.

3. Load Balancer vs API Gateway vs Reverse Proxy

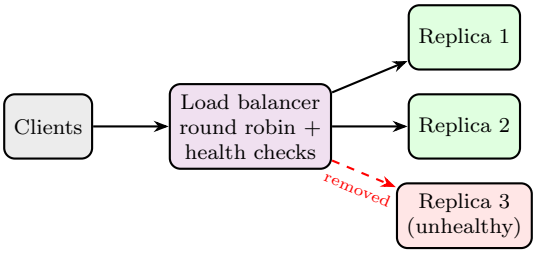
All three sit **between clients and servers** and receive requests on behalf of the backend, so they are often confused. The difference is **their main job**.

Reverse Proxy: a server that receives client requests and **forwards them to one or more backend servers**, hiding them from the outside. Clients only see the proxy. Typical jobs: TLS termination (handling HTTPS), caching static content, compression, hiding internal IPs, basic security. *Examples:* Nginx, HAProxy, Caddy, Envoy, Traefik.
(A *forward* proxy is the opposite: it acts on behalf of *clients*, e.g. a corporate proxy.)

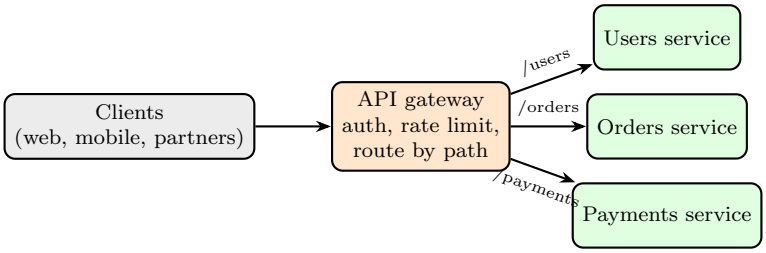


Load Balancer: distributes incoming traffic across **several identical replicas** of a service, so no server is overloaded and failed servers are removed automatically (health checks). It is a reverse proxy specialized in **distribution**.

- **Algorithms:** round robin, least connections, weighted, IP hash / consistent hashing (same client to the same server).
- **Layer 4** (TCP / UDP): routes by IP and port, very fast, does not read the request. **Layer 7** (HTTP): reads paths, headers and cookies to route.
- *Examples:* AWS ELB (ALB = L7, NLB = L4), Google Cloud Load Balancing, HAProxy, Nginx, Kubernetes Services.



API Gateway: the **single front door for many different APIs or microservices**, focused on **API management**. It routes each request to the right service by path or method (`/users` → users service, `/orders` → orders service) and applies cross-cutting policies: authentication (API keys, OAuth / JWT), rate limiting and quotas, request / response transformation, versioning, aggregation of several calls into one, analytics and billing. *Examples:* AWS API Gateway, Kong, Apigee, Azure API Management, Tyk.



	Reverse Proxy	**Load Balancer**	**API Gateway**
Main job	Front for servers: forward, hide, TLS, cache	Spread traffic over identical replicas	Manage many APIs: route, secure, meter
Routes to	One or a few backends	Many copies of the same service	Many different services
Layer	L7 (HTTP)	L4 or L7	L7, API aware
Key features	TLS termination, caching, compression	Algorithms, health checks, failover	Auth, rate limits, transformations, versioning, analytics
Examples	Nginx, Caddy, Envoy	AWS ALB / NLB, HAProxy	Kong, AWS API Gateway, Apigee

How they relate: a load balancer and an API gateway are both specialized reverse proxies, and one product can play several roles (Nginx or Envoy can do all three). A typical setup chains them: **Client** $\rightarrow$ **Load balancer** $\rightarrow$ **API gateway** $\rightarrow$ **services** (each service with its own internal load balancing). An **LLM gateway** is the same idea as an API gateway, specialized for model calls: routing between providers, token-based rate limits, cost tracking and semantic caching.